

Chapter # 8

Logical Modeling

It is about locating blue points by how data is accessed & organized.

↳ define structure of data elements & their relationships.
Conceptual modeling does not depend on the logical model.

These are three different logical models for representing multidimensional data structures:

Relational Model:

- o Used in Relational OLAP systems.

Multidimensional model:

- o Used in multidimensional OLAP systems.

Hybrid model:

- o Used in Hybrid OLAP Systems.

MOLAP Systems:

Data cubes:

- ⇒ A data cube concept introduced by Gray et al in 1977.
- ⇒ A data cube is a generalization of the SQL group-by set operator. It defines the space of basic and aggregate

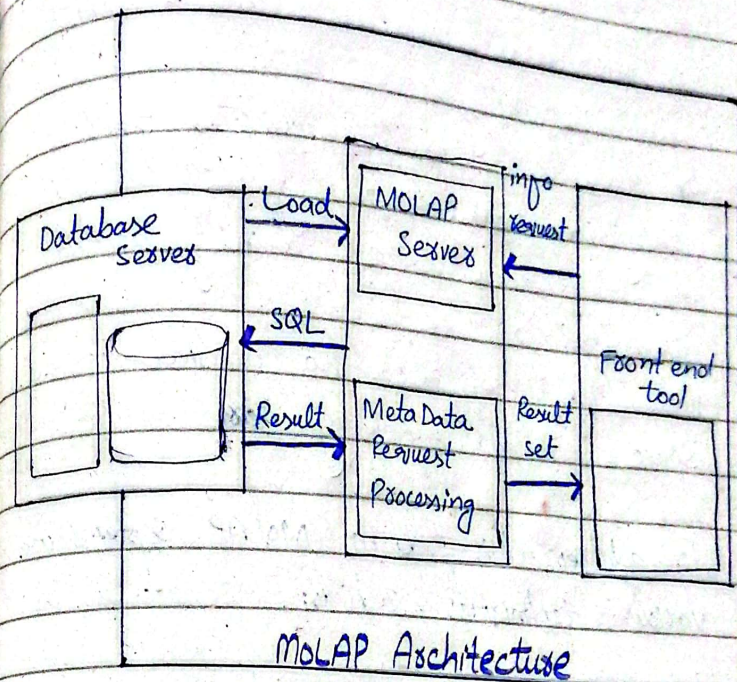
events that can be calculated and stored.

⇒ Given a primary cube, its data cube consists of the primary cube surrounded by a collection of secondary, coarser-grained cubes.

⇒ MOLAP is used for limited data volumes and in this data is stored in multidimensional array. In MOLAP, dynamic multidimensional view of data is created.

- MOLAP contains precalculated data sets.
- Contains all the possible answers to a given range of question.
- Fast query performance, due to optimized storage, multi-dimensional indexing.
- Automated computation of higher-level aggregates of the data.
- Limited flexibility.
- Slow loading
- Data redundancy.

MOLAP Architecture:



⇒ MOLAP Systems store all data cube cells, including non-informative cells, which can lead to sparsity and storage inefficiency.

⇒ The lack of standards for logical models hinders the widespread use of MOLAP systems.

The Problem of Sparsity

→ The main problem with MOLAP system is sparsity.

→ Data sparsity indicates that only a small part of the cells in multidimensional cube actually include information.

→ Sparsity can lead to wasted disk space and slower query performance.

To address this problem, MOLAP systems use various techniques, such as:

⇒ Selecting which part of the data cube to materialize. This involves deciding which secondary cubes to store physically and which to calculate on the fly. This can help balance storage space and query performance.

⇒ Partitioning cubes into smaller chunks. This can improve query performance by allowing chunks to be loaded into memory more quickly. It can also

help manage sparse chunks differently from dense chunks.

⇒ Compressing sparse chunks. This can reduce the amount of disk space used to store sparse chunks.

⇒ Indexing chunks for efficient access.

• These techniques can help MOLAP systems mitigate the problem of sparsity.

One of the problem with data cube is:

• Difficulty of maintaining the relationships between cells when rolling up or drilling down. This can slow down queries.

HOLAP Systems:

⇒ HOLAP systems use different strategies to store data in ROLAP and MOLAP modes.

⇒ These strategies include:

• Storing dense chunks in MOLAP mode and sparse chunks in ROLAP mode.

• Storing primary cubes in ROLAP mode and secondary cubes in MOLAP mode.

- Storing frequently accessed data in MOLAP mode and the remaining data in ROLAP mode.

ROLAP Systems

- ⇒ ROLAP system use the relational model to represent multidimensional data.
- ⇒ The relational model is a well known and widely used model in the database industry.
- ⇒ ROLAP systems are more scalable than MOLAP systems. Because, RDBMS are highly sophisticated and optimized tools that have been evolving years.
- ⇒ Relational model for multi-dimensional data is that it is a standard model that is familiar to database designers.

Multidimensional modeling in relational system:

- ⇒ Represented by:
 - Star schema
 - Snowflake schema

Star Schema:

⇒ Star schema is a common way to represent multidimensional data in relational databases.

structure of star schema:

- ⇒ It is composed of:
 - one fact table at center.
 - multiple dimensional tables.

Fact table:

- Also known as a reality table.
- It can be a combination of attributes from completely different dimensional table. can be used for analysis & decision making process.
- it forms a vertical table.
- it contains numbers, figures, matrix or quantitative data.
- Contains data on which different operations can be perform.

Dimensional table:

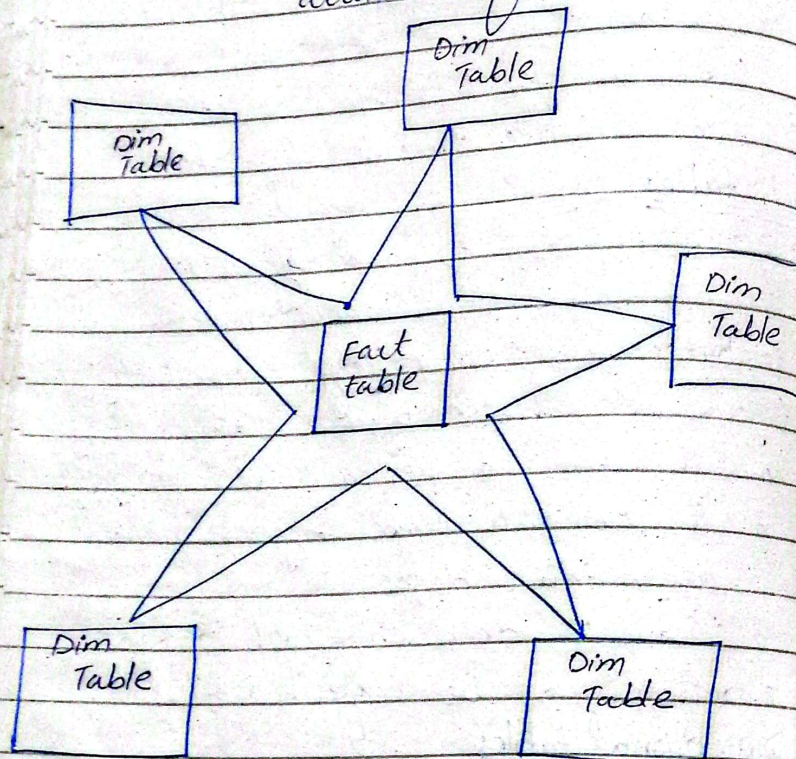
- A dimension table contains descriptive attribute fields.
- it contains more attributes in comparison to fact tables.

- The attribute format is in text form.
- it occurs before the fact tables.

Examples:

DimCustomers:

attributes of a customer.



Relationship:

⇒ Fact table and Dimension tables are related with PK & FK.

⇒ 1-to-M

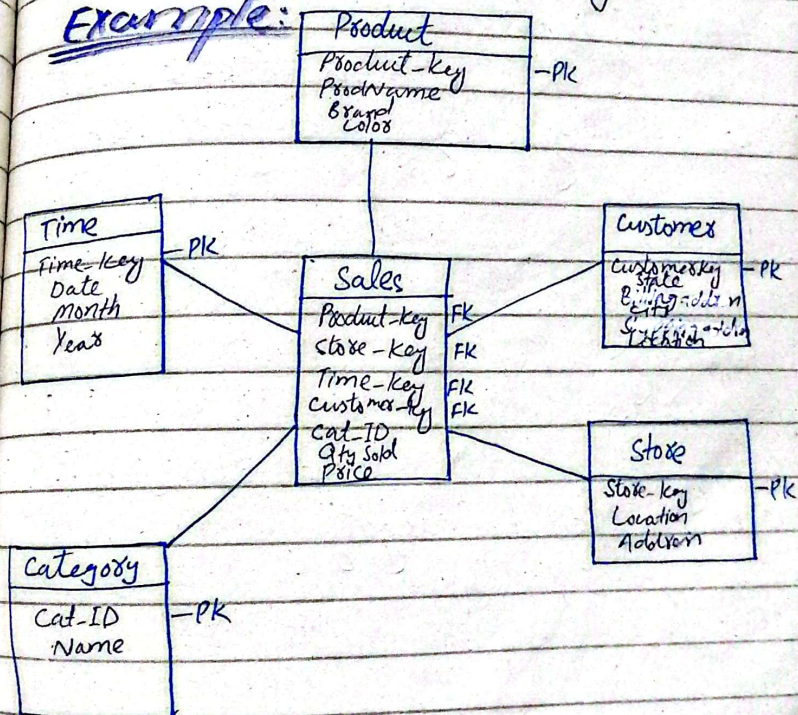
Transitive Dependencies:

⇒ Dimension tables are not in third normal form due to transitive dependencies. Means that there are attributes in the dimension tables that depend on other attributes in the same table which can lead to redundancy but improve query performance.

Data redundancy:

⇒ Redundancy in dimension tables can increase disk space usage.

Example:



Type of Keys in star schemas:

1. Primary Key:

- Each row in a dimensional table is identified by a unique value of attribute designated as the primary key of the dimension.

2. Surrogate Keys:

- The surrogate keys are simply system generated sequence numbers.
- No build in meaning.

3. Foreign Key:

- PK in a dimension table must be a foreign key in the fact table.

Pros:

- Simplicity
- Flexibility
- Scalability
- Query performance ↑ due to less joins.
- Built-in referential integrity.

Snowflake Schemas

- it is variation of star schema.
- Aim to remove redundancy.
- This is done to remove transitive functional dependencies.

Key points:

Normalization:

- Snowflake schemas partially normalize dimension tables to reduce redundancy and improve data integrity.

Granular Tables:

- Dimension tables are broken into smaller tables, more specific tables to represent different levels of details.

Surrogate Keys:

- Surrogate keys are often used in snowflake schemas to uniquely identify rows in dimension tables, which can improve performance & reduce storage.

Require less updates:

- Require less updates.

Increased Complexity:
→ Can be more complex to design and implement.

Cons:

Increased schema complexity:
Increased the number of tables and relationships can make the schema more complex to manage and understand.

Complex join operations:

Queries may require complex join, lead to slow down query performance.

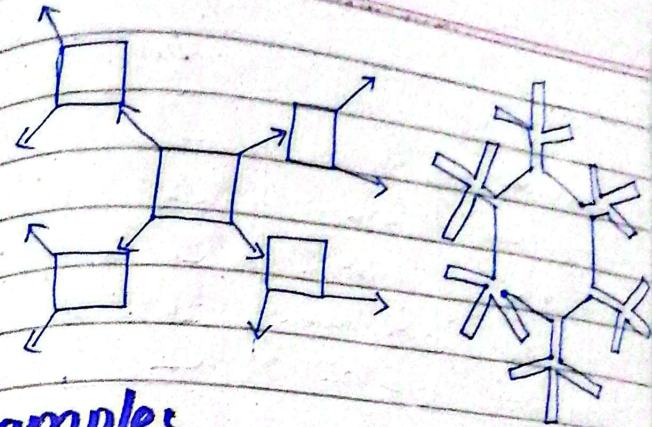
Increased storage overhead:

Snowflake schemas can reduce data redundancy, but they can increase storage overhead to the additional tables & indexes.

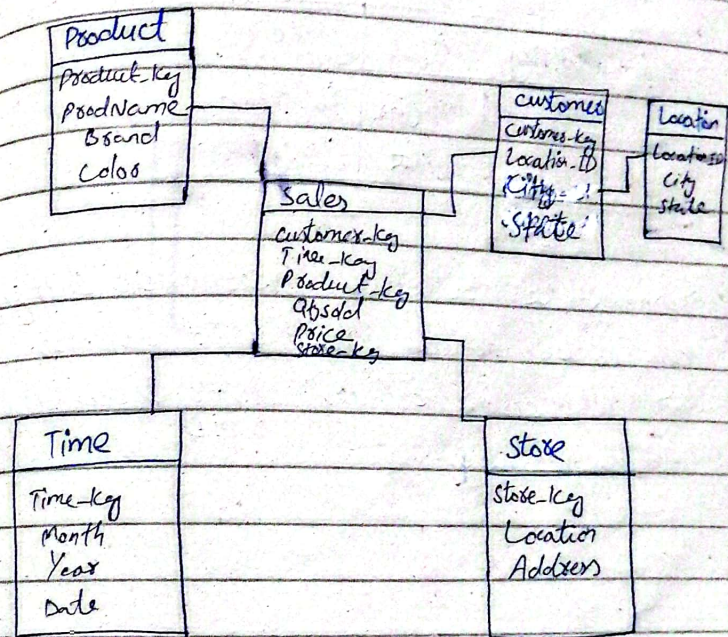
Structure:

⇒ Composed of:

- Fact table
- Dimension tables
- Sub-Dimension tables



Example:



Star VS Snowflake

Customer-Key	Customer-Name	Location-ID
C1	Amina	L1
C2	Hira	L2
C3	Harna	L1
C4	Khadija	L1
C5	Zada	L2

↓

Location ID	City	State
L1	Dalhousie	Pakistan
L2	Gurgaon	UP

⇒ Snowflake Schemata need new surrogate keys to be added in order to express the relationships between primary & secondary dimension tables.

Feature	Star Schema	Snowflake Schema
Structure	Simple, De-normalized design	Normalized design with multiple level of granularity
Data redundancy	Higher	Lower..
Query Performance	Generally faster due to fewer joins	Slower due to more complex joins
Storage space	Large	Smaller
Complexity	Simple to design & implement	more complex to design & implement.
Flexibility	Less flexible for complex data relationships.	more flexible for complex data relationships.
Best suited for	Simple data models & fast query performance	Complex data models and data integrity

Views

- ⇒ Data warehouse store a massive amount of data, making analysis difficult.
- ⇒ To simplify this, users often use selection and aggregation techniques.

Views:

- ⇒ Views are just tables that contain aggregated data.
- ⇒ They are categorized as:
 - Primary Views:
 - Represent the finest level of detail.

Secondary Views:

- Represent aggregated data.

Benefits:

Improved query performance:

- By precalculating and storing aggregated data.

Reduced Data Access:

- Reduce data access that need to be accessed & processed.

Simplified Analysis:

- Provide a simplified view of data.

Important Considerations when using views:

Data consistency:

- Ensure that the data is consistent in views with the underlying data source.

View maintenance:

- Regularly update views to reflect changes in the source data.

Storage Overhead:

- Storing views can increase storage requirements.

Calculating aggregate values:

- ⇒ When working with data warehouses, often need to calculate aggregate values. If aggregation operators is not used carefully, it can affect accuracy of results.

Distributive operators:

- ⇒ These operators can be applied to partially aggregated data.
e.g. SUM, MIN, MAX.

Non-Distributive operators:

⇒ These operators can lead to incorrect results if applied to partially aggregated data.
for example, AVG, COUNT.

Example:

type	Product	quantity	unitPrice	receipts
Dairy Product	Skim milk	5	1.0	5.0
Dairy Product	Fresh milk	7	1.5	10.5
Drinks	Coke	9	0.8	
Total:				22.7

SUM
↓

AVG
↓

Type	quantity	unitPrice	quantity x unitPrice
Dairy Product	12	1.25	15.0
Drinks	9	0.8	7.2
Total: 22.2			

Example: A few sales views that can be materialized:

$V_1 = \{ \text{Product, date, store} \}$

$V_2 = \{ \text{type, date, storeCity} \}$

$V_4 = \{ \text{type, month, state} \}$

$V_3 = \{ \text{category, month, storeCity} \}$

$V_5 = \{ \text{quarter, state} \}$

Holistic operators:

MEDIAN & MODE

Storing Aggregate Data in Relational Schemas:

⇒ Often need to store detailed and aggregated data in data warehouse. There are different ways:

1: Using a Single fact Table:

One approach is to store both detailed and aggregated data in a single fact table.

Differentiate between them using dimension table attributes.

for example:

Detailed data:

The dimension table attributes will have specific values (e.g. store name, product name, date).

Aggregated data:

The dimension table attributes will have NULL values or generic values (e.g. "All Stores", "All products", "All Years").

Example:

⇒ Have a sales fact table and a store dimension table.

⇒ You can store both individual sales & total sales by state in the same fact table.

SALES:

Keys	KeyP	KeyD	KeyC	quantity	receipts
1	1	1	2	170	85
2	1	1	1	300	150
3	...	...	...	1700	850

STORE:

Keys	store	storeCity	state	...
1	COOP1	Columbus	Ohio	...
2	-	Austin	Texas	...
3	-	-	Texas	...

Constellation schema for storing aggregate data:

⇒ It is an approach.

A constellation schema is a data design technique that uses multiple fact tables to store different levels of aggregated data.

⇒ This approach can improve query performance and reduce storage space compared to storing all data in a single, large fact table.

Multiple fact tables:

○ A constellation schema uses multiple fact tables, each containing a specific level of aggregated data.

Shared Dimension tables:

○ Dimension tables are shared across multiple fact tables, reducing data redundancy.

Improved query performance:

○ By storing only the relevant data in each fact table, queries can be optimized to access only the necessary information.

Reduced Storage space:

○ By eliminating redundant data.

Increased Complexity:

○ More complex to design & implement.

Temporal Scenarios

- ⇒ Often considers that dimensions are static but they are dynamic and can change over time.
- ⇒ For example, a product might change its category, a store might change its address.
- ⇒ These changes are called dynamic hierarchies.

Common Temporal Scenarios

Yesterday-for-Today:

- Using yesterday's data to analyze today's performance.

Today-for-Yesterday:

- Comparing today's data with yesterday's data.

Today-as-Yesterday:

- Analyzing data as either today or yesterday.

Today-and-Yesterday:

- Analyzing data for both today & yesterday.

Modeling Techniques for Dynamic Hierarchies

⇒ There are different techniques to model dynamic hierarchies in data warehouses.

Some common approaches include:

Slowly Changes Dimensions:

- There involves creating different types of SCDs to handle various change scenarios.

Effective Date and End Date:

- Adding effective date and end date columns to dimension tables to track changes over time.

Time-Series Fact Tables:

- Creating separate fact tables for different time periods.

Type-1 Dynamic Hierarchy (Today-for-Yesterday)

⇒ In this type of hierarchy, only consider the current state of the data.

⇒ Past changes are not tracked or considered.

⇒ When a change occurs in a dimension attribute, the old value is simply overwritten with the new one.

Limitations:

- Loss of historical Accuracy
- Limited flexibility

Use Cases:-

⇒ Best suited for data warehouse where the focus is on current and near-term analysis.

Type 2 - Dynamic Hierarchies: Yesterday-or-Today

⇒ This type allow to track changes in dimension attribute over time.

⇒ When a change occurs, a new record is added to the dimension table, preserving the historical information.

Pros:

- Preserve Historical Accuracy
- Flexible Analysis

Cons:

- Increased data volume
- Increased query complexity

Use cases:

⇒ For DWH that require detailed historical analysis & reporting.

Type-3 Dynamic Hierarchies:

Partial Logging

⇒ This approach is middle ground b/w Type-1 and Type-2.

⇒ It Allows to track changes to dimension attributes to a limited extent.

⇒ Store limited numbers of previous versions of an attribute, but not all historical values.

Pros:

- Reduced Storage
- Improved performance

Cons:

- Limited Historical Analysis
- Increased Complexity

Usecase:

⇒ Suitable when need to track some historical changes but do not require a full history.

Type-4: Full Data Logging

- ⇒ Most Comprehensive.
- ⇒ It involves storing every change to a dimension attribute, including the start and end dates of each change.
- ⇒ Allows for detailed historical analysis.

Pros:

- Detailed Historical Analysis
- Flexibility

Cons:

- Increased Storage
- Increased Complexity

Deleting Tuples:

⇒ Deleting data in DWH is not common, because often used to track changes over time.

⇒ However, to prevent databases from getting too big, should set a time limit for how long data is kept.

⇒ This limit is usually between three and five years.

⇒ Before deleting data, make sure there are no other tables that depend on it.